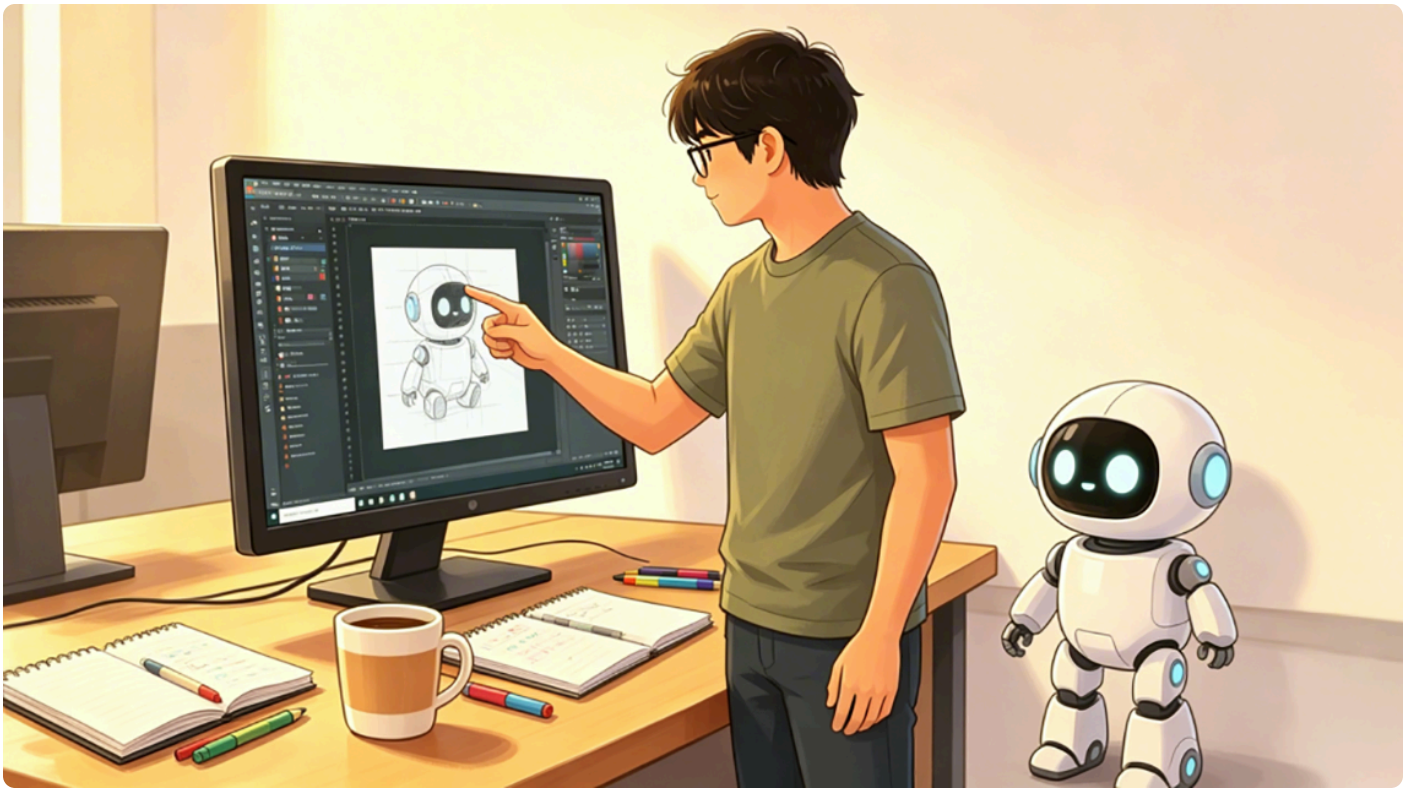
04 | 架构设计（上）：应用架构与代码组织

Robert · Claude Code 企业级全链路开发实战



你好，我是 Robert。

上一讲我们定义了 Hify 的产品边界和技术栈。这一讲回答一个更具体的问题：**代码怎么组织、外部调用怎么处理**？

很多工程师拿到需求直接开始写代码，写到一半发现模块之间耦合得死死的、线程池被 LLM 调用占满了管理页面打不开、不同模块的代码风格完全不一样。这些问题不是写代码的问题，是动手之前没想清楚应用架构。

这一讲和下一讲合起来是 Hify 的完整架构设计。这一讲（上篇）聚焦代码层面——模块怎么分、Spring 代码怎么组织、外部调用怎么处理。下一讲（下篇）聚焦系统层面——怎么部署、瓶颈在哪、未来怎么扩展、数据怎么存。

应用架构：模块化单体

第一个问题：一个 Spring Boot 应用，内部怎么组织？

第 03 讲已经确定了一期 50 人单机部署。但单机不等于所有代码堆在一个 package 里。我让 Claude Code 给方案：

> Hify 是一个 Spring Boot 单体应用，功能包括模型提供商管理、Agent 配置、对话引擎、知识库 RAG、简版工作流、MCP 工具接入。一个人开发，一期 50 人使用，但后续可能要扩到几千人。代码内部怎么组织？给我方案对比。

Claude Code 给了三个方案：

> 方案 A：按 package 分层。所有功能放在一个模块里，按 controller/service/mapper 分 package。最简单，但所有功能混在一起，后面代码量大了找文件费劲，想拆分也拆不动。
>
> 方案 B：Maven 多模块，按功能拆。一个 Spring Boot 应用，内部按 Maven 多模块组织，每个功能一个模块。模块之间通过接口调用，代码层面严格隔离。后续想拆分成微服务，只需要把接口调用改成远程调用，模块内部不用动。
>
> 方案 C：微服务。每个功能一个独立服务，通过网关路由，服务间 Feign 调用。

它推荐了方案 C，理由是方便独立扩展。

我追问：

> 一个人维护六七个微服务，每个要独立部署、独立配置、独立监控，精力消耗会不会太大？

它承认了，说一人开发建议方案 B。我的判断：**选方案 B，模块化单体**。一个人维护一个应用比维护六七个微服务轻松太多。但代码上做好模块隔离，后续需要拆分时成本可控。

最终的模块划分：

复制代码

```
hify/
├── hify-app/                # 启动模块，Spring Boot Application
├── hify-provider/           # 模型提供商管理
```

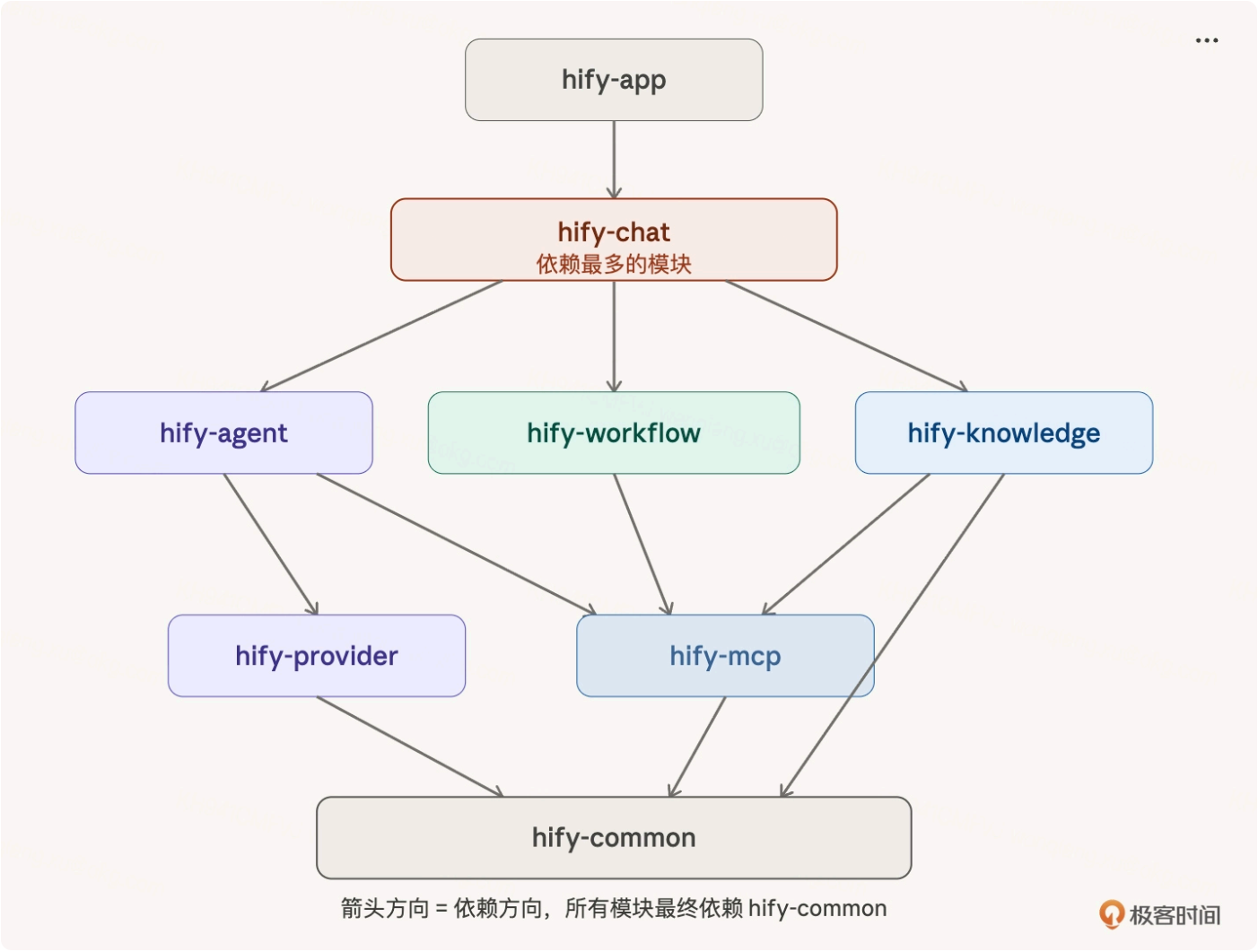
```
├── hify-agent/              # Agent 管理与配置
├── hify-chat/               # 对话引擎
├── hify-mcp/                # MCP 工具管理与调用
├── hify-workflow/           # 工作流编排与执行
├── hify-knowledge/          # 知识库与 RAG
├── hify-common/             # 公共模块（工具类、常量、异常、DTO）
├── hify-web/                # Vue 前端
└── deploy/                  # Docker Compose 配置
```

模块之间的依赖关系需要想清楚。我让 Claude Code 分析：

> 基于 Hify 的功能，帮我梳理这些模块之间的依赖关系。谁依赖谁？有没有循环依赖的风险？

它给的分析：hify-chat 是依赖最多的模块——对话时要读 Agent 配置（依赖 hify-agent）、调模型（依赖 hify-provider）、可能走工作流（依赖 hify-workflow）、可能做 RAG 检索（依赖 hify-knowledge）、可能调工具（依赖 hify-mcp）。hify-agent 依赖 hify-mcp（Agent 要绑工具）和 hify-knowledge（Agent 要绑知识库）。所有模块依赖 hify-common。

关键原则：**依赖是单向的，不能循环。hify-chat 可以依赖 hify-agent，但 hify-agent 不能反过来依赖 hify-chat。如果出现循环依赖，说明模块边界划错了，需要把共用的部分下沉到 hify-common**。



Spring 代码组织规范

这一节很多人觉得没必要讲，Controller、Service、Mapper 三层架构，谁不知道？

人写代码确实不需要讲。你凭经验和习惯，自然知道业务逻辑放 Service 里、Controller 只做参数校验。但 AI 不知道。你不告诉它规矩，它可能把事务管理写在 Controller 里、把数据库查询写在 DTO 里、直接引用另一个模块的 Mapper 把两个模块焊死。

这些规范不是写给人看的，是写给 Claude Code 看的。后面每一行代码都是它生成的，这些规矩定不好，生成出来的代码越多越乱。

我让 Claude Code 帮我梳理：

> Hify 是模块化单体，用 Spring Boot + MyBatis-Plus。帮我定义代码组织规范，覆盖：每个模块内部的分层结构、每一层的职责边界、跨模块调用的规则。要求具体到 AI 能直接执行，不要模糊的描述。

它给了一版，我调整后定稿（这里为了省篇幅，在一些较简单的场景就不贴 Claude 的输出了，后面在实操课统一演示），每个业务模块内部统一结构：

复制代码

```
src/main/java/com/hify/{module}/
├── controller/         # REST 接口
├── service/            # 业务逻辑接口
├── service/impl/       # 业务逻辑实现
├── mapper/             # MyBatis-Plus Mapper
├── entity/             # 数据库实体
├── dto/                # 请求/响应对象
├── config/             # 配置类
├── exception/          # 模块级自定义异常
└── constant/           # 模块级常量
```

每一层的职责边界：

> Controller 只做两件事：参数校验和调用 Service。不写业务逻辑、不做数据查询、不处理事务。为什么要这么严格？因为 Claude Code 特别喜欢在 Controller 里“顺手”加逻辑，它觉得方便，但你后面测试、重构、拆分全部受影响。
>
> Service 处理所有业务逻辑，包括事务管理、数据校验、业务规则。Service 之间可以互相调用，但只能调接口（interface），不能直接 new 实现类。
>
> Mapper 只做数据库操作。不要在 Mapper 的 XML 里写业务逻辑（比如复杂的条件判断），那是 Service 的事。
>
> Entity 和数据库表一一对应。DTO 是给接口用的请求 / 响应对象。Entity 和 DTO 之间要做转换，不要把 Entity 直接返回给前端——Entity 里可能有敏感字段（API Key），DTO 可以控制暴露哪些字段。

跨模块调用规则：**这条是模块化单体最关键的规范。模块之间只能通过 Service 接口调用，不能直接引用另一个模块的 Mapper、Entity 或内部类。**

为什么？**因为我们选模块化单体的一个重要理由是“后续能拆分”。**如果 hify-chat 直接引用了 hify-agent 的 Mapper，两个模块就焊死了——拆分时你要把所有这种引用找出来改成远程调用。但如果只通过 Service 接口调用，拆分时只需要把本地调用改成 HTTP/RPC 调用，接口签名都不用变。

我追问 Claude Code：

> 如果有一天要把 hify-chat 拆成独立服务，按照这套规范，改动量大概多大?

它分析了一下：Service 接口不变，实现类里的本地调用改成 Feign 调用，加上网络异常处理。每个跨模块调用点改两三行代码。改动量可控。

这就验证了这套规范的合理性。这里我标注下，能考虑到未来的拆分主要靠的是自己的项目经验，我希望你可以记下这个，就当积累一个经验。

外部调用设计

Hify 最大的技术挑战不在 CRUD，在外部 LLM 调用。第 03 讲运维预期已经指出：**瓶颈不在并发，在 LLM 长连接占用线程**。这一节把这个问题的解决方案定下来。

LLM API 调用有三个特点：慢（一次对话 3-30 秒）、不稳定（随时可能超时、限流、报错）、多供应商（每家行为不一样）。不提前设计好，上线就炸。

我让 Claude Code 系统性地分析：

> Hify 要调用多个外部 LLM API（OpenAI、Claude、Gemini、Ollama），这些调用慢且不稳定。从线程管理、容错、超时、重试四个维度，给出完整的技术方案。

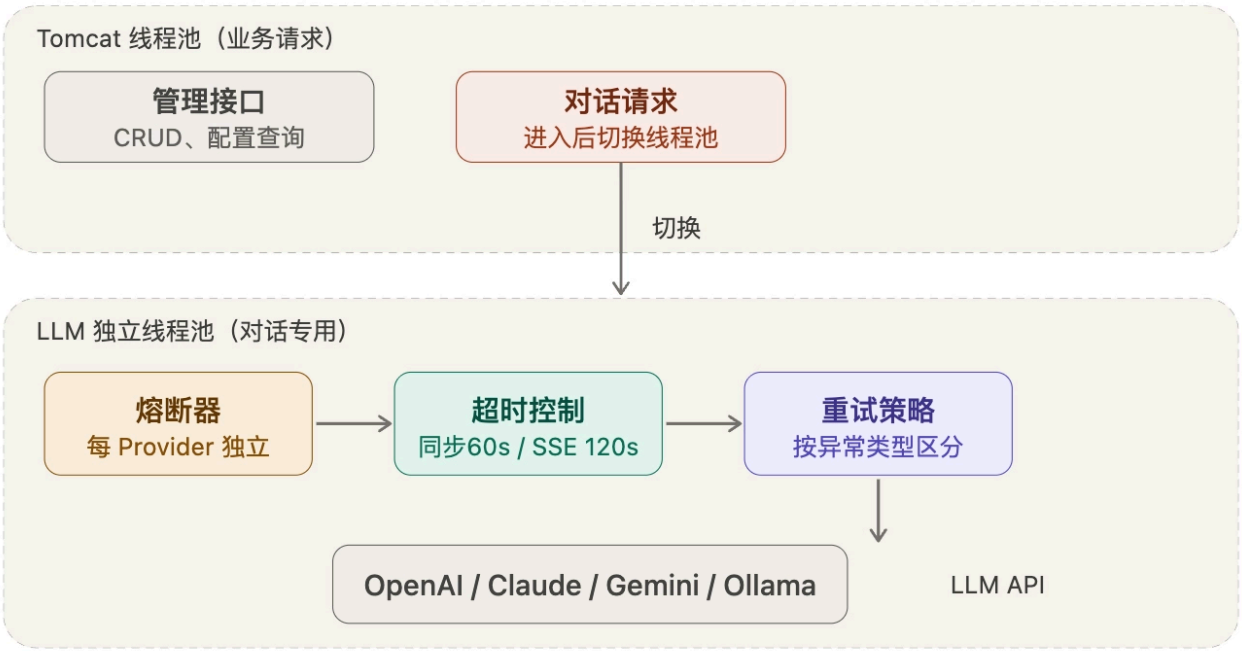
它给的方案很全面，我逐个判断：

1. 线程池隔离——同意。LLM 调用必须用独立线程池，不能和 Tomcat 的业务线程池共用。第 03 讲分析过：50 个并发 SSE 连接，每个持续 10–30 秒，如果共用线程池，管理页面的请求排在队列里等，用户看到的就是页面一直转圈。独立线程池解决这个问题——对话请求用自己的池子，管理请求用 Tomcat 默认的池子，互不影响。
2. 熔断机制——同意。某个 LLM 提供商连续失败时，快速熔断，后续请求直接返回错误，不浪费时间等超时。Claude Code 建议 Resilience4j，每个提供商独立熔断器——OpenAI 挂了不影响 Claude。合理。
3. 超时控制——同意，但我追问了细节。它建议对话接口 60 秒超时，连通性测试 10 秒。我追问：流式响应场景下，一个 SSE 连接可能持续一两分钟，60 秒会不会把正常对话切断？它调整了：SSE 连接超时设 120 秒，普通同步调用 60 秒。合理。
4. 重试策略——同意，细节有价值。不是所有失败都值得重试。网络抖动重试有意义，认证失败重试没用（重试一百次还是认证失败），限流需要退避重试（等一等再试）。按异常类型区分重试策略，这个细节 Claude Code 给得很好。

我又追问了一个它没主动提到的问题：

> 流式响应用 SSE，Spring MVC 怎么处理？需不需要引入 WebFlux？

它建议 WebFlux 做异步非阻塞。我不同意——第 03 讲已经确定不引入额外技术栈。SSE 场景用 Spring MVC 的 SseEmitter 就够，配合独立线程池和 120 秒超时，50 人完全能扛。不为了“技术上更优雅”引入一套新的编程模型。



极客时间

写进 CLAUDE.md

这一讲的所有决策写进 CLAUDE.md：

复制代码

```
## 架构设计

### 应用架构
模块化单体。一个 Spring Boot 应用，Maven 多模块组织。

模块划分：
- hify-provider：模型提供商管理
- hify-agent：Agent 管理与配置
- hify-chat：对话引擎
- hify-mcp：MCP 工具管理与调用
- hify-workflow：工作流编排与执行
- hify-knowledge：知识库与 RAG
- hify-common：公共模块

依赖原则：单向依赖，不循环。共用逻辑下沉 hify-common。

### 代码组织
每个业务模块统一结构：controller / service / mapper / entity / dto / config

分层规则：
- Controller 只做参数校验和调用 Service，不写业务逻辑
```

```
- Service 处理所有业务逻辑，包括事务管理
- 跨模块调用走 Service 接口，不直接引用其他模块的 Mapper 或 Entity
- Entity 不直接返回给前端，用 DTO 做转换

### 外部调用处理
- LLM 调用使用独立线程池，和业务请求隔离
- Resilience4j 熔断，每个提供商独立熔断器
- 同步调用 60s 超时，SSE 流式 120s 超时，连通性测试 10s
- 按异常类型区分重试：网络抖动重试、认证失败不重试、限流退避重试
- 流式响应使用 SseEmitter + 独立线程池，不引入 WebFlux
```

到这里，CLAUDE.md 包含了项目概述（第 03 讲）和应用架构（本讲）。Claude Code 对代码怎么写、放在哪、外部调用怎么处理已经有了完整的约束。

总结

这一讲做了三件事：确定了模块化单体的应用架构，定义了 Spring 代码组织规范，设计了外部调用处理方案。

回顾整个过程，最有价值的不是最终方案，而是和 Claude Code 的追问环节。它推荐微服务，被我用“一个人维护太重”否决了。它建议 WebFlux，被我用“不增加技术栈复杂度”否决了。它给的超时时间对 SSE 不够用，被我追问后调整了。

每一次追问的背后，都是你在用项目的真实约束来修正 AI 的通用建议。AI 给的是技术上最优的方案，你要基于约束选对你最优的方案。这两者经常不一样。

还有一点值得强调：代码组织规范看起来是常识，但对 AI 写代码来说是刚需。人凭习惯知道 Controller 不写业务逻辑，AI 不知道。你不告诉它，它就按自己的理解来，而且每次理解都不一样。这些规范现在花半小时定好，后面几百次代码生成都受益。

到这里，你可能会说我没有架构知识，不知道要问什么问题。那我想说，你以前没有，当看完这节课不就有了吗？你或者可以把这节课当作一个实践的架构课。记下来我们怎么思考的，以后就有经验了，知道怎么问问题了。

所以你在学这节课的时候得去思考我为什么这么问。而作为你自己，你会想到这么问吗？如果你不会这么问，那么就记住，以后这么问。这就是我说的方法论。

下一讲继续架构设计下篇——部署架构、性能瓶颈预判、扩展方案、数据模型。从代码层面进入系统层面。

思考题

看看你手头正在做的项目，问自己两个问题：

1. 你的模块之间是通过接口调用还是直接引用内部类？如果要把某个模块拆成独立服务，改动量大概多大？
2. 你的外部调用（数据库、第三方 API、消息队列）有没有做线程池隔离？如果某个外部服务卡了 30 秒，你的系统会怎样？

结合这两个问题让 Claude Code 帮你分析一下你当前项目的情况。

期待你的分享。如果今天的课程让你有所收获，也欢迎转发给有需要的朋友，邀请他来一起学习，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。 页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (17)

最新　精选



cricket1981
2026-04-12 来自上海

claude.md文件有长度限制吗？每次对话都会带上吗？哪些东西要放里面，哪些不应该放里面？期待老师解答一下

作者回复: 从体系来看，应该有三层：

1. claude.md
2. /docs 或/specs
3. skill

从一个大型的项目的角度来说，以我个人经验来看，合理的上下文应该是从这三个方面来组织的。

claude.md 应该放的是比如：项目简介和架构、技术栈版本、目录结构说明、开发规范（命名、格式）、常用命令、核心业务概念等信息

/docs 或 /specs 放的是详细的 API 设计文档、数据库 schema、业务流程文档等信息

skilll 放的应该是固定的流程说明等信息，比如CRUD。

回到本课程，因为这是一个从0开始的一个新项目。我的思路是从claude.md 到spec，再到skill，慢慢展开。

9

越学越菜，越菜越学

2026-05-30 来自四川

目前所有的东西都放在agents.md，等待后续开始实施以后，慢慢的前后端模块会堆积更多的内容，这个时候就拆到别的地方，skills，docs之类的，很多就不需要agents.md继续持有了

作者回复: 是的，就是到了合适的时候，就可以拆文档了。在前期规划太多文档，整理维护起来反而更累。

我习惯在前期只维护一个文档，再在合适的地方拆文档。反正都是AI干活，其实问题不大。适合自己的节奏就可以了。

2

Q:-_-

2026-04-07 来自广东

老师，课程上的图你都是用什么工具画的呀，想学习一下你的画图

作者回复: 我是用claude code画的，嘿嘿

共 3 条评论>

1

Geek_9ec8bf

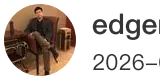
2026-04-05 来自广东

请问一下，在智能体开发这一块，我要如何做语言选型？到底是选Python呢，还是选TypeScript。Python的langchain跟langgraph生态发展由来已久，也很完善很成熟。TypeScript最近也非常火，特别是openclaw出来之后，它有很多杀手锏的这个应用，同时claude code也是用它来做的开发，还有open code。我们团队3个后端，两个前端，其中只有一个比较懂langchain的开发，想要问下做智能体开发，到底要如何选型？

作者回复: 我认为从几个方向考虑：
1. 从AI 辅助编程的角度，Python和TypeScript 效率是差不多的，都很成熟。
2. 智能体这块，python 的生态会比TypeScript更成熟。
3. 当前AI 领域TypeScript 目前更多是在Cli这块，也就是客户端应用。
所以，你说的智能体开发这个需求需要更细化。其实更主要的是看你们团队的语言倾向。长期单个语言解决不了所有问题。但是可以解决你们启动的问题。

凭我的直觉，既然你们是一个团队，后面应该需要server端服务。我个人会觉得python更适合一点。

1



edgen
2026-06-25 来自福建

“模块之间只能通过 Service 接口调用，不能直接引用另一个模块的 Mapper、Entity 或内部类。”，“不能直接引用另一个模块的 Mapper或内部类”这个容易做到，“不能直接引用另一个模块的 Entity”怎么做到呢？Service 接口返回的都是DTO吗？我目前的做法都是再Controller中将Service 接口返回的Entity转成DTO。

作者回复: ““不能直接引用另一个模块的 Entity”怎么做到呢”，这句话的意思是把Entity放在独立模块维护的意思。比如Entity 统一放在一个模块。

然后不散落在多个模块相互引用的意思。



terryking
2026-05-28 来自广东

它建议对话接口 60 秒超时， 这个应该是读写超时时间吧，不是SSE流的保持时间我理解, 所以只要SSE的连接一直都有socket的读写, 就不会断开，所以60秒其实是足够的；

作者回复: 是的，这个时间是超时时间，websocket/SSE在正常传输的时候，没有异常是不会断开的。60s 是足够的。

60指的是同步调用，连接的时候，不是数据传输的时间。






Dark
2026-05-23 来自江苏

是否需要引入DDD，进行领域驱动设,计

作者回复: 我这里没有引入DDD。个人觉的可以考虑，不强制。效果看个人对DDD的熟练度。从结果来看，不引入ddd，效果也是不错的。不影响结果。






snow.zhang
2026-04-19 来自江苏

本课程有源码分享吗?

作者回复: 有的，看课程介绍






黑鹰
2026-04-11 来自湖南

CLAUDE.md是讨论完最后生成，还是在讨论过程中边确认边写入?

作者回复: 我自己的习惯是，在最开始会有一个最基础的版本，也就是约束。然后边讨论边更新。因为功能需求，思考都是在开发过程中萌发的。然后自然而然会想把它拆为skill之类的相对高级的工具。比如后面的CRUD接口的开发流程。

共 2 条评论>





相见恨晚
2026-04-07 来自四川

老师好，能否给个搭建claude code这套开发环境的教程，就是跟你课程中的环境完全一致

作者回复: 我这套claude code开发环境没有特别的。只要安装上claude code 就可以。这里你可以去其他的课程里面学一下。